



IFSP SÃO CARLOS · ENGENHARIA DE SOFTWARE

Longest Substring Without Repeating Characters

Desafio Real de Processo Seletivo em Big Techs (LeetCode #3) — Otimização
Assintótica via Janela Deslizante (Sliding Window)

Rafael Feltrim

Desenvolvedor & Apresentador

Ian

Pesquisa & Algoritmos

Gustavo (Gub)

Análise de Complexidade

Onipresença nos Processos Seletivos

Onde é Aplicado?

Problema clássico no topo do ranking de entrevistas de alta escala:

Amazon (Top 10 SDE)

Meta (Triagem 45m)

Google (L3/L4)

Microsoft

Apple

Bloomberg

O Que é Avaliado?

- 1. Otimização de Força Bruta:** Sair de $O(N^3)$ para $O(N)$.
- 2. Escolha de Estruturas:** Uso de Hash Map para busca $O(1)$.
- 3. Gestão de Memória:** Uso de arrays diretos para tabela ASCII.
- 4. Edge Cases:** Strings vazias, únicas e repetidas.

Peso na Contratação

Em entrevistas técnicas, o código é apenas 30% da nota.

Os outros 70% medem:

Decomposição Lógica

Comunicação Clara

Justificativa Big-O

O Enunciado: Substring sem Duplicatas

Descrição do Problema

"Dada uma string s , encontre o comprimento da maior substring contígua que não contenha caracteres repetidos."

Distinção Crítica de Conceito:

Substring: Sequência contínua de caracteres (ex: "abc" em "abcde").

Subsequência: Mantém a ordem mas não é contínua (ex: "ace" em "abcde").

Exemplos de Teste

Exemplo 1:

$s = \text{"abcabcbb"} \rightarrow \text{Resposta: 3}$ (substring: "abc")

Exemplo 2:

$s = \text{"bbbbbb"} \rightarrow \text{Resposta: 1}$ (substring: "b")

Exemplo 3:

$s = \text{"pwwkew"} \rightarrow \text{Resposta: 3}$ (substring: "wke")

A Armadilha da Força Bruta $O(N^3)$

❌ Abordagem 1: Força Bruta

- 1. Gera todos os pares de início e fim (i, j) possíveis: $O(N^2)$ substrings.
- 2. Para cada substring, executa um loop interno para verificar se há duplicatas: $O(N)$.

Tempo: $O(N^3)$

Espaço: $O(\min(N, \Sigma))$

💀 **Resultado sob carga:** Para uma string de 50.000 caracteres, exige ~125 trilhões de operações → **Time Limit Exceeded (TLE)**.

🟢 Abordagem 2: Janela Deslizante (Sliding Window)

- 1. Mantém uma janela dinâmica delimitada por dois ponteiros: $[left, right]$.
- 2. O ponteiro $right$ expande a janela caractere por caractere.
- 3. Ao detectar duplicata, contrai a janela pela esquerda com um salto instantâneo!

Tempo: $O(N)$ Linear

Espaço: $O(\min(N, \Sigma))$

🚀 **Resultado:** Executa em passagem única em menos de **5 milissegundos!**

Dinâmica dos Dois Ponteiros & Hash Map

SLIDING WINDOW & TWO POINTERS ALGORITHM

VISUALIZING: Longest Substring Without Repeating Characters (LeetCode 3)

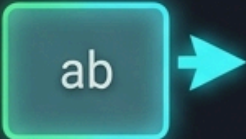
PROBLEM GOAL: Find the length of the longest substring with **NO** repeating characters.

['a' 'b' 'c' 'a' 'b' 'c' 'b' 'b']

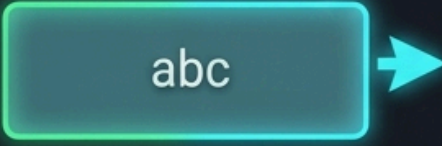


STEP 1: Start pointers L=0, R=0. Max Length = 1.

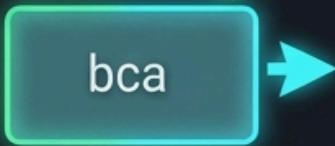
STEP 2: R moves to 'b'.
Window (L=0 to R=1):
Max Length = 2.



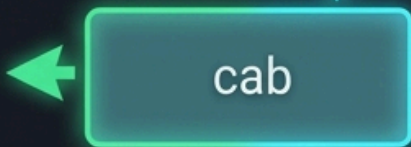
STEP 3: R moves to 'c'.
Window (L=0 to R=2):
Max Length = 3.



STEP 4: R moves to 'a' (Duplicate at index 3).
HashMap shows previous 'a' at index 0.
L shifts to index (0+1) = 1.
Window: "bca". Length = 3.



STEP 5: R moves to 'b' (Duplicate at index 4).
L shifts to previous 'b' index (1+1) = 2.
Window: "cab". Length = 3.



And so on...

CHARACTER lastIndexMap	
Character	Last Seen Index
'a'	0
'b'	1
'c'	2
'a'	3
'b'	4, ...

$L = \max(L, \text{map.get(char_at_R)} + 1)$
when conflict occurs.

Current Max Length:
'a' ['?']

Current Max Length: 3
Window Content

🔑 A Mecânica da Otimização

1. Ponteiro Direita (right):

Percorre a string da esquerda para a direita inserindo novos caracteres na janela.

2. Tabela Hash (char_map):

Armazena a relação $\text{map}[\text{caractere}] = \text{ultimo_indice}$ em tempo constante $O(1)$.

3. Salto do Ponteiro Esquerda (left):

Ao achar caractere já visto na janela, salta direto para $\text{map}[\text{char}] + 1$ sem varredura intermediária!

Implementação Otimizada em $O(N)$

Python (Legibilidade e Expressividade)

```
def lengthOfLongestSubstring(s: str) -> int: char_map = {} # char -> último índice
esquerda = 0 maior = 0 for direita, char in enumerate(s): # Se o caractere já está na
janela ativa: if char in char_map and char_map[char] >= esquerda: esquerda =
char_map[char] + 1 # Pulo  $O(1)$  char_map[char] = direita maior = max(maior, direita
- esquerda + 1) return maior
```

C Baixo Nível (Array Direto ASCII)

```
int lengthOfLongestSubstring(char* s) { int last_index[256]; // Tabela direta  $O(1)$ 
for (int i = 0; i < 256; i++) last_index[i] = -1; int max_len = 0, left = 0, n = strlen(s);
for (int right = 0; right < n; right++) { unsigned char c = (unsigned char)s[right]; if
(last_index[c] >= left) { left = last_index[c] + 1; // Salto em  $O(1)$  } last_index[c] = right;
int cur = right - left + 1; if (cur > max_len) max_len = cur; } return max_len; }
```


Análise Rigorosa de Complexidade (Big-O)

ABORDAGEM ALGORÍTMICA	COMPLEXIDADE DE TEMPO	COMPLEXIDADE DE ESPAÇO	CLASSIFICAÇÃO EM ENTREVISTA
1. Força Bruta (Loops Aninhados)	$O(N^3)$	$O(\min(N, \Sigma))$	Rejeitado (Red Flag)
2. Sliding Window com Set (Remoção Passo a Passo)	$O(2N) = O(N)$	$O(\min(N, \Sigma))$	Contratado (Hire)
3. Sliding Window com Hash Map (Pulo $O(1)$)	$O(N)$ Estrito	$O(\min(N, \Sigma))$	Forte Candidato (Strong Hire)

Justificativa de Tempo: $O(N)$

O ponteiro direita avança de 0 até N-1 exatamente uma vez. Cada consulta, inserção e cálculo de máximo leva tempo constante $O(1)$.

Justificativa de Espaço: $O(\min(N, \Sigma))$

O espaço ocupado pelo Hash Map é limitado pelo menor valor entre o comprimento da string N e o tamanho do alfabeto Σ (ASCII = 256).

Casos de Borda & Perguntas da Banca

Casos de Borda Validados

1. String Vazia "": Retorna 0 imediatamente.
2. Caracteres Idênticos "bbbbbb": A janela se mantém em tamanho 1.
3. Todos Distintos "abcdef": A janela expande até o tamanho total N.
4. Espaços e Símbolos "a b c!": Tratados nativamente pela tabela ASCII.



Obrigado a Todos!

Abrimos agora para dúvidas do professor e da turma.

[Código no GitHub](#)[Vault Obsidian Integrado](#)